Working with the MISP 2.5 REST API

Query, interpret and automate

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

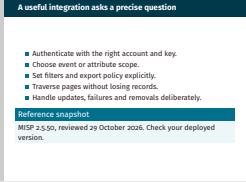
MISP
INTELLIGENCE SHARING
Working with the MISP 2.5 REST API
Query, interpret and automate
CIRCL / MISP Project
https://www.misp-project.org/
Reference snapshot: 29 October 2026

A useful integration asks a precise question

- Authenticate with the right account and key.
- Choose event or attribute scope.
- Set filters and export policy explicitly.
- Traverse pages without losing records.
- Handle updates, failures and removals deliberately.

Reference snapshot

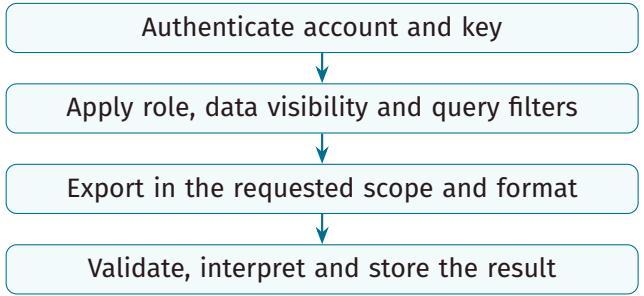
MISP 2.5.50, reviewed 29 October 2026. Check your deployed version.

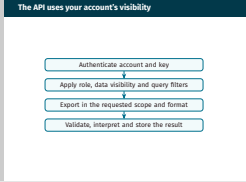


Allow 40–50 minutes for the presentation and another 30–40 for the workshop. Learners need basic HTTP and JSON knowledge. This replaces a lesson about the original API redesign with current operational contracts. The README pins MISP and PyMISP source revisions; patch levels, permissions, settings and exporters affect behaviour.

Make one request

The API uses your account's visibility





An API key is not a bypass around organisation, distribution or sharing group visibility. A site-admin test alone does not verify an analyst-facing integration. In the lab, an ordinary user in the owning org saw five synthetic domain attributes while a user in a different org saw only its own private fixture. API operation permission and data visibility are separate checks.

Set up a key for the task

- The account needs API access: `perm_auth`.
- Send the raw key in `Authorization`.
- Advanced keys can restrict expiry, source IP and writes.
- Keep keys outside scripts, notebooks and shared collections.

Read-only does not mean GET-only

A read-only key can POST a search, while event creation is denied.

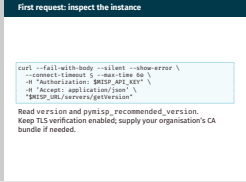
Set up a key for the task
The account needs API access: perm_auth.
Send the raw key in Authorization.
Advanced keys can restrict expiry, source IP and writes.
Keep keys outside scripts, notebooks and shared collections.
Read-only does not mean GET-only
A read-only key can POST a search, while event creation is denied.

There is no Bearer prefix in the normal MISP Authorization header. AuthKey restrictions are combined with the user's role and data visibility. Generate and manage keys through the profile/auth-key UI under your instance's policy. Advanced keys are displayed once. The live test used an expiring read-only key for an ordinary User, successfully POSTed searches and received HTTP 403 when attempting event creation.

First request: inspect the instance

```
curl --fail-with-body --silent --show-error \
  --connect-timeout 5 --max-time 60 \
  -H "Authorization: $MISP_API_KEY" \
  -H 'Accept: application/json' \
  "$MISP_URL/servers/getVersion"
```

Read `version` and `pymisp_recommended_version`.
Keep TLS verification enabled; supply your organisation's CA bundle if needed.



Set URL and key locally before the command; never paste a working key into the slides. MISP_URL is the base URL including any deployment subdirectory and without a trailing slash for these curl examples. CURL_CA_BUNDLE and REQUESTS_CA_BUNDLE can supply private CA trust. Check 4xx, redirects, login HTML and server errors before trying a broad data export. The Python sample allows plaintext HTTP only for an isolated loopback lab.

Choose the endpoint by the result you need

Endpoint	Result
`/events/index`	Event summaries
`/events/restSearch`	Full events or metadata
`/attributes/restSearch`	Matching attributes
`/events/view/{uuid}`	One known event

Use POST with a JSON body for search. Read each endpoint's contract.

Choose the endpoint by the result you need
Endpoint Result
/events/index Event summaries
/events/restSearch Full events or metadata
/attributes/restSearch Matching attributes
/events/view/{uuid} One known event
Use POST with a JSON body for search. Read each endpoint's contract.

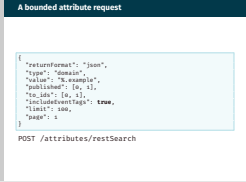
The first three endpoints support the POST search patterns taught here; event view is a GET. Current restSearch also supports objects, sightings and galaxy clusters, each with different parameters and formats. Do not claim every MISP endpoint uses one universal query or response schema. Deployment prefix handling matters when building URLs.

A bounded attribute request

```
{
  "returnFormat": "json",
  "type": "domain",
  "value": "%.example",
  "published": [0, 1],
  "to_ids": [0, 1],
  "includeEventTags": true,
  "limit": 100,
  "page": 1
}
```

```
POST /attributes/restSearch
```


A bounded attribute request



The complete file also requests event UUIDs and a stable ID order. Reserved .example names make the fixture recognisable. An empty result is correct before creating training data. These raw JSON flag arrays include both states deliberately. Set Accept and Content-Type to application/json. The example does not request attachments or broad enrichment data.

The response envelope depends on the scope

Attribute search

```
{"response": {"Attribute": [
  {"uuid": "...", "type": "domain", "value": "..."}
]}}
```

Event search

```
{"response": [
  {"Event": {"uuid": "...", "info": "..."}}
]}
```




The ellipses are structural illustrations, not runnable fixture UUIDs. Event index instead returns a flat summary array, and event view returns an Event object. Validate the expected envelope before consuming it. PyMISP removes the outer response wrapper and can construct typed objects, so raw HTTP parsers must not be reused on pythonified results.

Search with intent

A matching event can contain other attributes

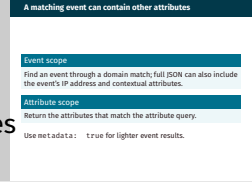
Event scope

Find an event through a domain match; full JSON can also include the event's IP address and contextual attributes.

Attribute scope

Return the attributes that match the attribute query.

Use `metadata: true` for lighter event results.



The live fixture had domain and ip-dst attributes. An event search filtered by type domain still returned the contextual IP in its full JSON. This is useful for analysis but matters when constructing a detection feed. Metadata is a deliberate reduced representation, not a complete event with no attributes. Include the scope in downstream data contracts.

Exact values and wildcard values are different

Filter value	Intent
`workshop.example`	Exact attribute value
`%.example`	Suffix match
`%REST API training%`	Event-info substring

Start with one known value or UUID. Add filters one at a time.

Exact values and wildcard values are different

Filter value	Intent
`workshop.example`	Exact attribute value
`%.example`	Suffix match
`%REST API training%`	Event-info substring

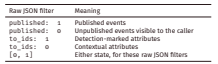
Start with one known value or UUID. Add filters one at a time.

The event-info query initially returned zero when given only REST API training; the explicit wildcard form matched. The published example uses that tested form. These are supported SQL-style wildcard patterns, not a regex language. Exact/wildcard behaviour is field-specific; consult the deployed endpoint when extending the examples.

Select publication and detection flags explicitly

Raw JSON filter	Meaning
`published: 1`	Published events
`published: 0`	Unpublished events visible to the caller
`to_ids: 1`	Detection-marked attributes
`to_ids: 0`	Contextual attributes
`[0, 1]`	Either state, for these raw JSON filters

Select publication and detection flags explicitly



Raw JSON filter	Meaning
`published: 1`	Published events
`published: 0`	Unpublished events visible to the caller
`to_ids: 1`	Detection-marked attributes
`to_ids: 0`	Contextual attributes
`[0, 1]`	Either state, for these raw JSON filters

Use numeric 0 and 1 in these search examples. A literal false can skip the filter in the reviewed pipeline, so it is not universally equivalent to requesting zero. Create/edit payloads have their own boolean handling. PyMISP 2.5.34.4 rejects a to_ids list despite the raw server accepting it. For its JSON non-restrictive search, omitting to_ids includes both states. The raw live tests verified unpublished contextual attributes with numeric zeros.

Express tag logic with explicit operators

```
{
  "tags": {
    "AND": ["tlp:clear"],
    "OR": ["training:reviewed", "training:verified"],
    "NOT": ["training:exclude"]
  }
}
```

Require the AND tag, one OR tag, and no NOT tag.

Express tag logic with explicit operators
{
"tags": {
"AND": ["tlp:clear"],
"OR": ["training:reviewed", "training:verified"],
"NOT": ["training:exclude"]
}
}
Require the AND tag, one OR tag, and no NOT tag.

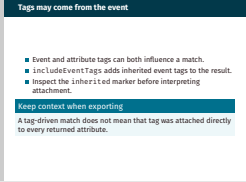
This illustrative combination requires the custom training tags to exist and be attached; it is not the supplied fixture's default query. The live tests separately verified AND, OR and NOT selection using the event's tlp:clear tag. Do not copy dashboard-specific include/exclude substring syntax into a restSearch tags object. A query filter is not an access-control policy.

Tags may come from the event

- Event and attribute tags can both influence a match.
- `includeEventTags` adds inherited event tags to the result.
- Inspect the `inherited` marker before interpreting attachment.

Keep context when exporting

A tag-driven match does not mean that tag was attached directly to every returned attribute.



The JSON exporter marks included EventTag-derived tags as inherited. The lab verified that event-level tlp:clear selected both domain attributes and appeared with the inherited marker. Event-scope matching and attribute result selection have different granularity. Event-tag changes also matter for incremental consumers: an attribute timestamp alone need not describe every parent-context change.

Pick the clock that answers your question

Field	Clock
`timestamp`	Change time in the selected scope
`event_timestamp`	Parent event change time
`publish_timestamp`, `last`	Event publication time
`from`, `to`	Event date bounds
`first_seen`, `last_seen`	Observation time

Pick the clock that answers your question

Field	Clock
`timestamp`	Change time in the selected scope
`event_timestamp`	Parent event change time
`publish_timestamp`, `last`	Event publication time
`from`, `to`	Event date bounds
`first_seen`, `last_seen`	Observation time

For attributes, timestamp and the explicit attribute_timestamp refer to attribute change time. Event timestamp is a separate field. Relative publication windows do not substitute for an update cursor, and observation time is not ingestion time. The range helper accepts inclusive lower and upper bounds. Use fixed Unix timestamps across a paginated run instead of recalculating a relative window on each page.

Changing the format can change the selection

JSON / CSV	Detection text
Non-restrictive exporters in this snapshot. State selection explicitly when comparing results.	Defaults to published events and detection-marked attributes when those filters are omitted.

An analysis result is not automatically a deployment policy.

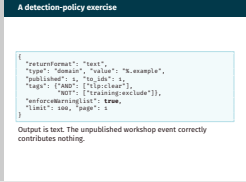
Changing the format can change the selection
JSON / CSV
Non-restrictive exporters in this snapshot. State selection explicitly when comparing results.
Detection text
Defaults to published events and detection-marked attributes when those filters are omitted.
An analysis result is not automatically a deployment policy.

MispAttribute::restSearch injects published=1 and to_ids=1 when the exporter is restrictive. JsonExport and CsvExport declare themselves non-restrictive. TextExport does not. Defaults may vary by format and source revision. Explicit filters, warninglists, decay and handling restrictions deserve review before feeding a defensive system.

A detection-policy exercise

```
{
  "returnFormat": "text",
  "type": "domain", "value": "%.example",
  "published": 1, "to_ids": 1,
  "tags": {"AND": ["tlp:clear"],
           "NOT": ["training:exclude"]},
  "enforceWarninglist": true,
  "limit": 100, "page": 1
}
```

Output is text. The unpublished workshop event correctly contributes nothing.



The file under examples contains the full request. The published lab fixture had a detection-marked domain and a contextual domain; only the detection-marked one appeared in this text export. The .example restriction keeps the exercise synthetic. Never deploy these fixture values as a real blocklist. Other formats such as STIX and IDS rules have their own compatibility and representation contracts.

Warninglist annotations and filtering differ

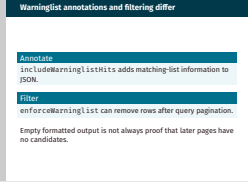
Annotate

`includeWarninglistHits` adds matching-list information to JSON.

Filter

`enforceWarninglist` can remove rows after query pagination.

Empty formatted output is not always proof that later pages have no candidates.



fetchAttributes applies warninglist enforcement after fetching rows and can skip them. A short or empty output page may therefore be misleading as a completion signal. The supplied pagination sample rejects post-query filter options and non-JSON formats rather than claiming one loop safely traverses every exporter. For those formats, inspect the deployed exporter and its count/skipped semantics and design a dedicated traversal.

Automate and verify

A role cap can make every page short

Requested	Returned	Continue?
Page 1, limit 100	2	Yes
Page 2, limit 100	2	Yes
Page 3, limit 100	1	Yes
Page 4, limit 100	0	Stop for this plain JSON query

Five records were available. Stopping at the first short page loses three.

A role cap can make every page short

Requested	Returned	Continue?
Page 1, limit 100	2	Yes
Page 2, limit 100	2	Yes
Page 3, limit 100	1	Yes
Page 4, limit 100	0	Stop for this plain JSON query

Five records were available. Stopping at the first short page loses three.

This is the actual tested role-cap fixture: the ordinary User role had restsearch_limit_result set to two in the disposable lab. The event cap uses a scope-specific divisor, so it differed from the attribute cap. Do not request a higher-privilege key just to avoid pagination. The loop's empty-page rule is intentionally limited to the sample's supported JSON filters; it is not a universal export rule.

Bound the traversal and reject incomplete results

- Start at page 1; request a deterministic order.
- Validate the envelope and attribute UUIDs.
- Continue through short pages.
- Fail on repeated UUIDs or a maximum-page bound.
- Publish output and checkpoints only after success.

Bound the traversal and reject incomplete results
- Start at page 1; request a deterministic order.
- Validate the envelope and attribute UUIDs.
- Continue through short pages.
- Fail on repeated UUIDs or a maximum-page bound.
- Publish output and checkpoints only after success.

The example forces Attribute.id ASC and permits 1–1000 for page size and maximum pages, with defaults 100 and 20. A repeated UUID can indicate a repeated page or concurrent drift; it is treated as incomplete, not silently skipped. Stable sort does not create a database snapshot and does not solve every concurrent deletion or visibility change. X-Result-Count on restSearch is this render's count, not a universal global total or cursor.

Use the supplied read-only Python client

```
python examples/search_attributes.py \
  --page-size 100 --max-pages 20 \
  > attributes.ndjson.tmp \
  && mv attributes.ndjson.tmp attributes.ndjson
```

- Environment-provided URL and key; trusted TLS.
- Timeouts, no redirects, JSON/envelope checks.
- A failure leaves partial output unpublished.

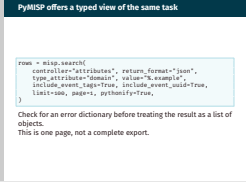
Use the supplied read-only Python client
python examples/search_attributes.py \
--page-size 100 --max-pages 20 \
> attributes.ndjson.tmp \
&& mv attributes.ndjson.tmp attributes.ndjson
Environment-provided URL and key; trusted TLS.
Timeouts, no redirects, JSON/envelope checks.
A failure leaves partial output unpublished.

Install the pinned requirements in a virtual environment first. The client's supported filter set is deliberately narrow and excludes attachment downloads and post-query warninglist/decay filtering. It does not save a checkpoint. Protect the temporary file as intelligence output. Requests keeps normal proxy and CA handling. Errors do not dump keys, headers or server response bodies. This is a teaching client, not a replacement for a production ingestion service or PyMISP.

PyMISP offers a typed view of the same task

```
rows = misp.search(
    controller="attributes", return_format="json",
    type_attribute="domain", value="%.example",
    include_event_tags=True, include_event_uuid=True,
    limit=100, page=1, pythonify=True,
)
```

Check for an error dictionary before treating the result as a list of objects.
This is one page, not a complete export.



The companion constructs PyMISP with environment configuration, TLS verification and timeouts, and validates a list of MISPAttribute instances. Current client parameter names differ from raw JSON: type_attribute rather than type, return_format rather than returnFormat. Unknown kwargs are not a substitute for checking the server contract. Empty lists are valid. Do not use ExpandedPyMISP in the new lesson or assume list index zero exists.

Keep one fixed window for an incremental run

1. Capture an upper timestamp at run start.
2. Set the lower bound from the last successful run, with overlap.
3. Fetch every page using the same inclusive bounds.
4. Upsert by UUID and complete downstream writes.
5. Advance the checkpoint only when the whole run succeeds.

Keep one fixed window for an incremental run
1. Capture an upper timestamp at run start.
2. Set the lower bound from the last successful run, with overlap.
3. Fetch every page using the same inclusive bounds.
4. Upsert by UUID and complete downstream writes.
5. Advance the checkpoint only when the whole run succeeds.

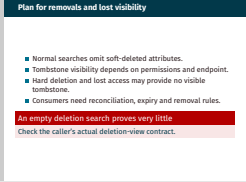
A JSON timestamp range is [lower, upper], with inclusive comparisons. Choose a documented first-run backfill and overlap policy. Duplicate records across runs are expected with overlap and should be reconciled by UUID and version/change time. This is a design pattern, not an implemented guarantee in the supplied client. It does not capture every parent tag, distribution, access-loss or hard-deletion change; reconciliation remains necessary.

Plan for removals and lost visibility

- Normal searches omit soft-deleted attributes.
- Tombstone visibility depends on permissions and endpoint.
- Hard deletion and lost access may provide no visible tombstone.
- Consumers need reconciliation, expiry and removal rules.

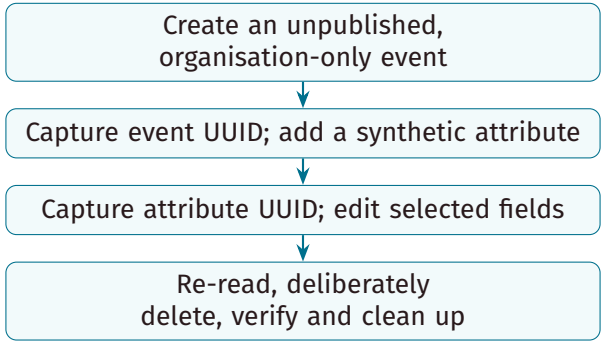
An empty deletion search proves very little

Check the caller’s actual deletion-view contract.



In this source, attribute restSearch deleted=1 returned a tombstone to a sync-capable administrator but not the ordinary owning user. That owner could inspect the deleted record through events/view/UUID/deleted:1. fetchAttributes has a sync-permission condition for the numeric deleted option. Do not grant sync privileges just for the workshop or promise that any ordinary reader receives all removals. Use MISP server synchronisation for its intended purpose rather than replacing it with a naive polling loop.

Write a small, explicit change



Write a small, explicit change
Create an unpublished, organisation-only event
Capture event UUID; add a synthetic attribute
Capture attribute UUID; edit selected fields
Re-read, deliberately delete, verify and clean up

The JSON files and README implement this sequence with no fixed IDs. The event omits date so the server supplies today's date. Distribution zero is organisation-only; the attribute inherits its event with distribution five. It is contextual and correlation-disabled for the exercise. Publishing is separate and can trigger sync or alerts; the learner's event stays unpublished. Do not run write exercises against production data.

Update only the fields you intend

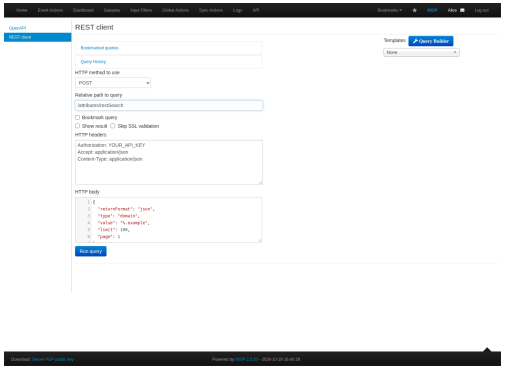
```
{
  "Attribute": {
    "comment": "Reviewed in the REST API workshop.",
    "to_ids": false
  }
}
```

`POST /attributes/edit/{attribute-uuid}` Re-read the attribute and verify the comment.



Do not replay the whole fetched event or attribute just to change one comment. It may include stale timestamps, distribution and unrelated data. Reconcile after a conflict. A timed-out write may have succeeded: blind retries can create duplicates or overwrite newer state. UUID identity helps reconcile operations, but is not universal idempotency for every endpoint. The lab verified this edit through an ordinary writer's own new attribute.

Use the built-in tools to inspect a request



- `/api/rest`
- Templates and query builder.
- History and bookmarks.
- Generated request code.

Use the built-in tools to inspect a request



- `/api/rest`
- Templates and query builder.
- History and bookmarks.
- Generated request code.

This genuine 2.5.50 capture shows a prepared request with the visible YOUR_API_KEY placeholder and no submission. See the full-size README image for labels. The older servers/rest path redirects here. Review generated curl/Python before reuse: remove embedded keys, add timeouts and keep TLS verification. The deployed OpenAPI UI is at api/openapi.

Try the read-only notebook and Postman collection

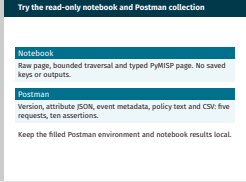
Notebook

Raw page, bounded traversal and typed PyMISP page. No saved keys or outputs.

Postman

Version, attribute JSON, event metadata, policy text and CSV: five requests, ten assertions.

Keep the filled Postman environment and notebook results local.



The committed environment has a blank secret api_key and a placeholder base URL. The actual collection was executed with Newman against the lab's read-only key. It checks the response status and relevant envelope/type; each request is a single page. The notebook cells never create or delete records. Mutation is deliberate through the separate workshop instructions. Older notebooks, collections and executable samples live only in the archive.

Classify failure before retrying

Symptom	Next check
401 / 403	Key, expiry, IP, role and endpoint permissions
404	Base path, UUID, existence and caller visibility
400 / 405	Method, JSON shape, types and validation
HTML / redirect	Login or proxy page; do not treat as empty data
429 / 5xx / timeout	Bounded read retry; reconcile uncertain writes
200, no rows	Scope, flags, tags, clocks, ACL and exporter defaults

Classify failure before retrying

Symptom	Next check
401 / 403	Key, expiry, IP, role and endpoint permissions
404	Base path, UUID, existence and caller visibility
400 / 405	Method, JSON shape, types and validation
HTML / redirect	Login or proxy page; do not treat as empty data
429 / 5xx / timeout	Bounded read retry; reconcile uncertain writes
200, no rows	Scope, flags, tags, clocks, ACL and exporter defaults

Invalid-key and read-only-write tests returned 403 in the reviewed lab; do not teach one universal status for all authentication failures. Some failures also arrive as JSON error dictionaries, especially through PyMISP, so status alone is insufficient. Keep response diagnostics local and protect them as exported intelligence. Retry safe reads with bounded backoff; do not convert failure to an empty successful export.

Workshop: prove the boundaries

1. Create the private fixture and capture its UUIDs.
2. Query it as the owning organisation’s reader.
3. Compare another organisation and a read-only key.
4. Traverse the role-capped pages.
5. Edit, re-read and deliberately soft-delete the attribute.
6. Inspect the owner deletion view; remove the lab event.

Workshop: prove the boundaries
1. Create the private fixture and capture its UUIDs.
2. Query it as the owning organisation’s reader.
3. Compare another organisation and a read-only key.
4. Traverse the role-capped pages.
5. Edit, re-read and deliberately soft-delete the attribute.
6. Inspect the owner deletion view; remove the lab event.

The README gives the exact JSON files, endpoint sequence and curl headers. Use normal User privileges for the owning fixture; installation administration is not required. A trainer can impose a small role result cap in an isolated lab to reproduce the short-page case, but changing a shared production role is outside the exercise. Use reserved domains and documentation IPs, no executable samples or live incident records. Retire lab keys after use.

Evidence an integration reviewer can reproduce

- Fourteen offline client-boundary tests.
- Live MISP checks for ACLs, paging, filters and writes.
- Newman assertions and an executable read-only notebook.
- Pinned MISP/PyMISP sources and documented limits.

Continue from the repository

See `a.7-rest-API/README.md` for commands, source audit and the complete workshop.

Evidence an integration reviewer can reproduce
Fourteen offline client-boundary tests.
Live MISP checks for ACLs, paging, filters and writes.
Newman assertions and an executable read-only notebook.
Pinned MISP/PyMISP sources and documented limits.
Continue from the repository
See a.7-rest-API/README.md for commands, source audit and the complete workshop.

Offline tests substitute HTTP responses to check the client, not MISP. Live tests used real MariaDB, Valkey and MISP 2.5.50 with two organisations. They covered five visible domains despite a two-row role cap, read-only searches and denied write, tag logic/inheritance, flags/time bounds, envelopes, text/CSV, typed PyMISP and the write/deletion sequence. Do not present this as exhaustive coverage of every MISP API or deployment.